

James Hyle
CS450: Programming Language Translation
Dr. Sherman
3/13/26

Milestone 2: ANTLR Project: Building Visitors

What didn't work:

I had a pretty hard time with this assignment; I underestimated the amount of work and research that it would take. (I spent quite a bit of time reading documentation at <https://help.eclipse.org/latest/index.jsp?topic=%2Forg.eclipse.jdt.doc.isv%2Freference%2Fapi%2Forg%2Feclipse%2Fjdt%2Fcore%2Fdom%2FForStatement.html> as well as <https://docs.oracle.com/javase/specs/jls/se6/html/j3TOC.html>) I think I implemented the first checkpoint with all test cases passing in about 10-12 hours, but in this assignment, I worked for probably 25 hours total and I am still not passing 15 tests. I was getting pretty confused by the JDT AST structure and how it compares to the grammar we are working with. Not all of the names have a one-to-one relationship for example JDT has PrimitiveType, SimpleType, and ArrayType under three separate classes while our grammar collapses them into a single rule type. It was hard for me to figure out initially how the visitors should construct the separate types and I am still not 100% positive on their relationship.

What did work:

In the previous project I went about the work by tackling the smallest pieces first, the exp tree, incdec statements, and types, and this went relatively smoothly as well as some other small pieces. I didn't have to restart after trying at first which was a great improvement compared to the first checkpoint. I did end up using AI to help me understand some of the concepts we dealt with and to check myself and my understanding of what is happening "under the hood". I think it was very beneficial for me to use it as it helped me clear up a couple of concepts that I was not grasping at all. For example connecting the JDT-AST builder to its antlr generated visitors was not clicking for me but realizing that we are writing code to connect our grammar to the JDT representation really helped get things going. That is ExtendedStaticJavaASTBuilder is using the classes inside the Eclipse JDT kit to create the actual program's AST.

What I learned:

I learned a lot about using Antlr again in this checkpoint, I am a big fan of the rule labels as it makes things much easier to deal with after Antlr generates the parser. I like that we can specify those labels as it helps me remember them greatly. About midway through I would say I kind of got stuck in the weeds dealing with so many different rules and labeling helped me sort them out. I also have been reflecting on the concept of double dispatch and its examples in the parser with the visit() and accept() methods. We are able to route each method call to its proper place in a very elegant way that eliminates if else chains and its polymorphism is one of my favorite aspects of this project so far. We have not covered the visitor pattern in CS472 OO Design Patterns, but this has been a cool learning experience with them so far. Learning about them is very cool to me because many things in programming and software engineering follow explicit patterns, the hard part is figuring out where they go and how they fit together with each other.